

L03 — Asymptotic Notations: Big-O, Big-Ω, Big-Θ with Examples

Unit: 1 | Chapter: Fundamentals of Data Structures & Arrays | BT Level: BT4 | CO: CO1

Learning Objectives

- Define Big-O, Big-Ω, and Big-Θ notations mathematically
- Relate asymptotic notations to real-number comparisons using mathematical analogies ($\leq$, $\geq$, $=$, $<$, $>$)
- Identify the asymptotic complexity of algorithms using each notation
- Simplify expressions using asymptotic rules
- Determine tight bounds for common algorithms

1. Why Asymptotic Analysis?

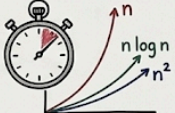
When we say an algorithm takes $3n^2 + 5n + 7$ operations, the dominant term for large n is n^2 . Constants and lower-order terms become insignificant.

Asymptotic analysis describes the growth rate of a function as $n \rightarrow \infty$, ignoring:

- Constant coefficients (hardware-dependent)
- Lower-order terms (irrelevant for large n)

2. Mathematical Analogy of Asymptotic Notations

To build an intuitive understanding, comparing two growth functions $f(n)$ and $g(n)$ is analogous to comparing two real numbers a and b :

ALGORITHM ANALYSIS: ASYMPTOTIC NOTATION ANALOGY			
NOTATION TYPES (e.g., $f(n)$ vs. $g(n)$)		MATHEMATICAL ANALOGY (a vs. b)	
	$f(n)$ is $O(g(n))$ $f(n)$ is $\Omega(g(n))$ $f(n)$ is $\Theta(g(n))$ $f(n)$ is $o(g(n))$ $f(n)$ is $\omega(g(n))$	$a \leq b$ $a \geq b$ $a = b$ $a < b$ $a > b$	 
* Definition: Mathematical methods to classify algorithm based on how run-time (or space) grows with input size.		* Pros: Simple to compare and contrast algorithm efficiency. (Independent of hardware, as stated before but contextualized).	
* Key idea: Focuses on the rate of growth, not exact numbers.		* Example 1: $f(n) = 3n^2 + 5n$ is $\Theta(n^2)$ because $3n^2$ dominates.	
* Cons: Hides constant factors. Only relevant for *large* input size.		* Example 2: $f(n) = n$ is $O(n^3)$, an upper bound.	
* IN SHORT: These notations provide a quick way to compare growth rates, not exact performance on specific hardware.		* Example 3: $f(n) = 5n$ is $\Omega(n^{1/2})$, a lower bound.	

2.1 Notation vs. Real Number Analogy

Asymptotic Notation (\$f(n)\$ vs. \$g(n)\$)	Mathematical Analogy (\$a\$ vs. \$b\$)	Intuitive Meaning	Bound Type
$f(n) \text{ is } O(g(n))$	$a \leq b$	$f(n)$ grows at most as fast as $g(n)$	Upper Bound
$f(n) \text{ is } \Omega(g(n))$	$a \geq b$	$f(n)$ grows at least as fast as $g(n)$	Lower Bound
$f(n) \text{ is } \Theta(g(n))$	$a = b$	$f(n)$ grows at the exact same rate as $g(n)$	Tight Bound
$f(n) \text{ is } o(g(n))$	$a < b$	$f(n)$ grows strictly slower than $g(n)$	Strict Upper Bound
$f(n) \text{ is } \omega(g(n))$	$a > b$	$f(n)$ grows strictly faster than $g(n)$	Strict Lower Bound

2.2 Key Characteristics & Examples

- **Definition:** Mathematical methods to classify algorithms based on how run-time (or space) grows with input size.
- **Key Idea:** Focuses on the **rate of growth**, not exact operational counts.
- **Pros:** Simple to compare and contrast algorithm efficiency independently of specific hardware or operating systems.
- **Cons:** Hides constant factors; only relevant for *large* input sizes (n).

Examples:

1. **Tight Bound (Θ):** $f(n) = 3n^2 + 5n$ is $\Theta(n^2)$ because $3n^2$ dominates for large n .
2. **Upper Bound (O):** $f(n) = n$ is $O(n^3)$ because $n \leq n^3$ for $n \geq 1$ (valid upper bound, though not tight).
3. **Lower Bound (Ω):** $f(n) = 5n$ is $\Omega(n^{1/2})$ because $5n \geq n^{1/2}$ for $n \geq 1$.

In Short: Asymptotic notations provide a quick, hardware-independent way to compare growth rates rather than exact execution time on a specific device.

3. Big-O Notation — Upper Bound

Definition: $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Interpretation: $g(n)$ is an **upper bound** on the growth of $f(n)$. The algorithm runs **at most** this fast.

3.1 Examples

$f(n)$	Big-O	Reasoning
--------	-------	-----------

$3n + 5$	$O(n)$	$3n + 5 \leq 4n$ for $n \geq 5$
$2n^2 + 3n$	$O(n^2)$	$2n^2 + 3n \leq 3n^2$ for $n \geq 3$
5	$O(1)$	constant, no growth
$n \log n + n$	$O(n \log n)$	$n \log n$ dominates
$n^3 + 2^n$	$O(2^n)$	2^n dominates for large n

3.2 Formal Verification Example

Prove that $f(n) = 4n + 2$ is $O(n)$:

Choose $c = 5, n_0 = 2$:

- For $n \geq 2$: $4n + 2 \leq 4n + n = 5n$ ✓

So $c = 5, n_0 = 2$ satisfies the definition.

4. Big-Ω Notation — Lower Bound

Definition: $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Interpretation: $g(n)$ is a **lower bound** on the growth of $f(n)$. The algorithm takes **at least** this long.

4.1 Examples

$f(n)$	Big-Ω	Reasoning
$3n^2 + 1$	$\Omega(n^2)$	$3n^2 + 1 \geq 3n^2$ for all $n \geq 0$
$n \log n$	$\Omega(n)$	$n \log n \geq n$ for $n \geq 2$
$n!$	$\Omega(2^n)$	$n!$ grows faster than 2^n

4.2 Use Case

Ω notation is used to show **algorithmic lower bounds** — for example, any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons in the worst case.

5. Big-Θ Notation — Tight Bound

Definition: $f(n) = \Theta(g(n))$ if:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0$$

Interpretation: $g(n)$ is a **tight bound**. The algorithm grows **exactly as fast** as $g(n)$ (up to constant factors).

$$f(n) = \Theta(g(n)) \text{ if and only if } f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

5.1 Examples

$f(n)$	Big- Θ	Reasoning
$2n^2 + 5n$	$\Theta(n^2)$	Bounded above and below by n^2
$n/2$	$\Theta(n)$	Bounded above and below by n
$\log_2(n) + 3$	$\Theta(\log n)$	Bounded above and below by $\log n$

5.2 Formal Verification Example

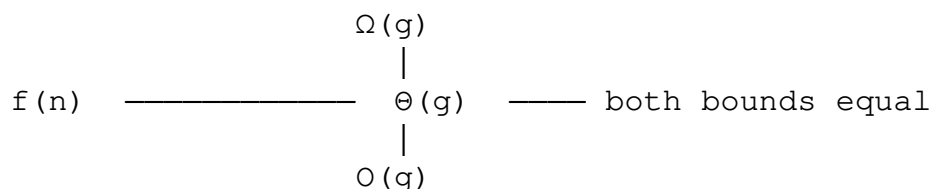
Prove $f(n) = 2n + 3$ is $\Theta(n)$:

- **Upper bound:** $2n + 3 \leq 3n$ for $n \geq 3$, so $f(n) = O(n)$ with $c_2 = 3$
- **Lower bound:** $2n + 3 \geq 2n$ for all $n \geq 0$, so $f(n) = \Omega(n)$ with $c_1 = 2$

Since both hold: $f(n) = \Theta(n)$ ✓

6. Summary of Notations

Notation	Meaning	Use Case
$O(g)$	Upper bound — "at most"	Worst-case analysis
$\Omega(g)$	Lower bound — "at least"	Best-case analysis, proving lower bounds
$\Theta(g)$	Tight bound — "exactly"	Average/typical behavior



7. Asymptotic Simplification Rules

Rule 1: Drop Constants

$$O(5n) \rightarrow O(n), O(100) \rightarrow O(1)$$

Rule 2: Drop Lower-Order Terms

$$O(n^2 + n) \rightarrow O(n^2), O(n \log n + n) \rightarrow O(n \log n)$$

Rule 3: Dominant Term

$$O(n^3 + n^2 + n) \rightarrow O(n^3)$$

Rule 4: Logarithm Base

$$O(\log_2 n) = O(\log_{10} n) = O(\ln n) \text{ — all equivalent (differ by constant factor)}$$

Rule 5: Sum Rule

If $T_1 = O(f)$ and $T_2 = O(g)$, then $T_1 + T_2 = O(\max(f, g))$

Rule 6: Product Rule

If $T_1 = O(f)$ and $T_2 = O(g)$, then $T_1 \times T_2 = O(f \times g)$

8. Worked Examples

Example 1: Nested Loop

```
for i in range(n):          # O(n)
    for j in range(n):      # O(n)
        print(i, j)         # O(1)
# Total:  $O(n \times n \times 1) = O(n^2)$ 
```

Example 2: Sequential Loops

```
for i in range(n):          # O(n)
    print(i)

for j in range(n):          # O(n)
    for k in range(n):      # O(n)
        print(j, k)
# Total:  $O(n) + O(n^2) = O(n^2)$  [Sum Rule → take max]
```

Example 3: Halving Loop

```
i = n
while i > 0:                # How many times can we halve n before reaching 0?
    i = i // 2              # i = n, n/2, n/4, ..., 1 →  $\log_2(n)$  iterations
# Total:  $O(\log n)$ 
```

Example 4: Identify complexity of $f(n) = 6n^3 + 4n^2 + 2n + 100$

- Dominant term: n^3
 - Big-O: $O(n^3)$, Big-Ω: $\Omega(n^3)$, Big-Θ: $\Theta(n^3)$
-

9. Little-o and Little-ω (Brief Overview)

- **$o(g)$** : Strictly less than g — f grows **strictly slower** than g (e.g., $n = o(n^2)$)
- **$\omega(g)$** : Strictly greater than g — f grows **strictly faster** than g (e.g., $n^2 = \omega(n)$)

These are used less frequently but appear in theoretical analysis.

Summary

Notation	Bound Type	Formal Condition	Mathematical Analogy
$O(g)$	Upper	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$	$a \leq b$
$\Omega(g)$	Lower	$f(n) \geq c \cdot g(n)$ for $n \geq n_0$	$a \geq b$
$\Theta(g)$	Tight	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	$a = b$
$o(g)$	Strict Upper	$f(n) < c \cdot g(n)$ for all c	$a < b$
$\omega(g)$	Strict Lower	$f(n) > c \cdot g(n)$ for all c	$a > b$

- Drop constants and lower-order terms.
- Use the dominant term to determine complexity class.
- Big- Θ is the most precise — use it when both bounds match.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 3 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 2.2 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 2.1 (Springer, 2024)